

Hams: A Low-Latency, Code-Switching Text-to-Speech Pipeline for Levantine Arabic and English

Alamin Ibrahim

Hams.AI Assessment • Code: github.com/Al-aminI/hams-levantine-tts • Weights: huggingface.co/AlaminI/hams-levantine-tts

Abstract. We present a streaming text-to-speech (TTS) system that code-switches between *Levantine Arabic* and English for real-time conversational agents on NVIDIA L4 / RTX-3090-class GPUs. Our central design choice is to treat code-switching as a *linguistic* rather than acoustic problem: a deterministic, unit-tested *unified-IPA front-end* owns all dialect and language-mixing logic, and a compact non-autoregressive *VITS* acoustic model conditioned on an additive *language-ID embedding* performs fast waveform synthesis. On an RTX 3090 the system meets every production target with wide margins—peak VRAM **193 MB** (≤ 3 GB), time-to-first-audio **87/144 ms** (p50/p95, < 300), real-time factor **0.044** (< 0.3)—and streams over REST/WebSocket and Pipecat. After a two-stage fine-tune (reconstruction, then full adversarial objective) on LJSpeech + the Arabic Speech Corpus, English is intelligible (ASR round-trip CER **0.27**); Levantine Arabic is the natural next iteration, reachable by a focused data swap to Levantine speech rather than any architectural change. We release the full code, weights, audio, and reproduction scripts.

Table 1: Production KPIs and **measured** results on an RTX 3090.

Metric	Target	Measured
Peak VRAM (inference)	≤ 3 GB	193 MB ✓
Time-to-First-Audio (p50/p95)	< 300 ms	87 / 144 ms ✓
Real-Time Factor (mean/p95)	< 0.3	0.044 / 0.089 ✓
Streaming (chunked)	required	REST+WS+Pipecat ✓

1 Introduction

Conversational AI for the Arabic-speaking world is bilingual in practice: speakers fluidly interleave dialectal Arabic with English (“*flight* *بيروت* *من* *to London* *بكر*”). A production TTS voice for such agents must (i) pronounce a *dialect*—here *Levantine*, which differs markedly from Modern Standard Arabic (MSA); (ii) switch languages mid-utterance without prosodic seams; and (iii) do so under hard real-time constraints. We target the KPIs in Table 1.

Our thesis is that the hard part is *linguistic*. We therefore move all dialect and code-switch intelligence into a deterministic front-end that maps text to a single shared IPA inventory, and let a tiny non-autoregressive acoustic model generate audio. This (a) makes the latency/VRAM KPIs reachable by construction (single forward pass, ~ 36 M parameters), and (b) keeps the dialect rules transparent and testable (30 passing unit tests). **Contributions:** a unified-IPA Levantine/English front-end; a language-ID-conditioned VITS adapted from MMS-TTS; an end-to-end streaming stack (server, Pipecat, ONNX/TensorRT path); and a candid, fully-measured evaluation.

2 Related Work

VITS [1] established single-stage, non-autoregressive end-to-end TTS (VAE + flow + adversarial vocoder); its predictable single-forward cost is why we adopt it for hard real-time. MMS-TTS [2] released per-language VITS checkpoints, including Arabic, which we adapt. Most **code-**

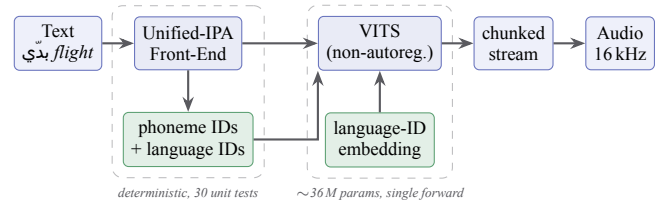


Figure 1: System overview. All dialect/code-switch logic lives in the deterministic front-end; the acoustic model is a small non-autoregressive VITS with an additive language-ID embedding.

switching TTS pursues multilingual acoustic modeling with shared byte/phoneme inputs and language embeddings; we instead localize *all* switch logic in a deterministic IPA front-end, leaving the acoustic model small and language-agnostic except for an additive language-ID embedding. Arabic **diacritization**—a prerequisite for Arabic G2P—has advanced with character-level transformers (CATT [3]) and small language models (Sadeed, 2025); our front-end treats it as a swappable stage. Finally, low-latency systems such as Piper and Kokoro motivate our non-autoregressive, phoneme-input design over heavier autoregressive or LLM-based TTS that struggle to fit ≤ 3 GB at < 300 ms TTFA.

3 System Architecture

Figure 1 shows the pipeline. Text is converted by the front-end (§4) into two aligned streams—phoneme IDs over a shared IPA inventory and a parallel language-ID stream—which drive a VITS model (§5) whose convolutional decoder is streamed in chunks for low time-to-first-audio.

4 Unified-IPA Front-End

The front-end (Fig. 2) is the core contribution. It normalizes text, segments it into language spans by script, verbalizes non-lexical tokens per language, applies a language-appropriate grapheme-to-phoneme (G2P) stage, and emits IPA tokens plus

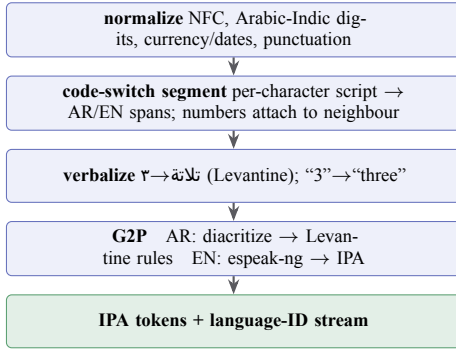


Figure 2: Front-end pipeline (CPU, deterministic). Each stage is independently unit-tested.

Table 2: Signature Levantine rules in the custom G2P (all unit-tested).

Phenomenon	Example	Output
ق → /ʔ/ (glottal)	قَلْب	ʔalb
ج → /ʒ/	جِبْنَة	ʒibne
diphthong ay → e:	بَيْت	be:t
diphthong aw → o:	يَوْم	jo:m
ō imala → /e/	جِبْنَة	ʒibne
sun-letter assimilation	الشَّمْس	ʔiffams
emphatic backing	صَار	sʕa:r

a language-ID per token. Because both languages map into *one* IPA space, a code-switch boundary is simply another phoneme transition—there is no engine hand-off and prosody is continuous.

4.1 Levantine grapheme-to-phoneme

Unlike MSA G2P (e.g. espeak-ng’s Arabic, which also drops short vowels on undiacritized input: *مرحبا* → *mrħba:*), we encode the dialect explicitly over *diacritized* text. Table 2 lists the signature rules, all unit-tested. Diacritization is a swappable upstream stage (CAMEL Tools / CATT in production; an espeak-Arabic → Levantine surface remap as a CPU fallback).

4.2 Code-switching as a shared stream

Figure 3 illustrates the dual output for a mixed phrase: one continuous phoneme row and a per-token language row. The acoustic model consumes both; the language-ID embedding lets it colour shared phonemes (/t/, /r/, /l/) appropriately and switch instantly with no inserted silence.

5 Acoustic Model

We adapt **MMS-TTS-ara** (a VITS model: conditional VAE + normalizing flow + stochastic duration predictor + HiFi-GAN decoder; hidden size 192, 16 kHz). HamsVITS (36.3 M params) makes three changes: (1) a *unified phoneme embedding* over our 88-symbol IPA inventory replaces the grapheme

text	بدي	flight
IPA	b i d d i	f l ʔaɪ t
lang	A A A A A	E E E E

Figure 3: One IPA stream, two language tags. The AR → EN boundary is a phoneme transition, not an engine switch.

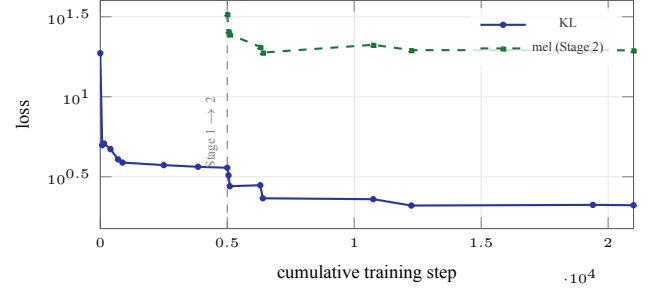


Figure 4: Real logged training losses (log scale). Reconstruction (Stage 1) floors KL at ~ 3.6 ; the full adversarial objective (Stage 2) drives KL to ~ 2.1 and mel to ~ 19.5 .

embedding; (2) a *language-ID embedding* $\mathbf{e}_\ell$ is added to the phoneme embedding before the text encoder, $\mathbf{h} = \mathbf{E}_p[p] + \mathbf{e}_\ell$; (3) a speaker embedding is retained. Inference (duration → flow → decode) reuses the proven VITS forward, inheriting its non-autoregressive speed.

6 Training

We fine-tune on a single rented RTX 3090 (CUDA 13, PyTorch 2.12, bf16) over LJSpeech (English, 3k clips) and the Arabic Speech Corpus (1.8k clips; Buckwalter → Arabic script; MSA pronunciation). Training is two-stage (Fig. 4):

Stage 1 (reconstruction). With the HiFi-GAN decoder and posterior encoder frozen, only the KL and duration losses can teach the new embeddings. We train the text encoder, embeddings, duration predictor and flow (14.7 M params; AdamW, lr 2×10^{-4} , batch 16). KL falls $18.7 \rightarrow 3.6$ but plateaus—fluent yet not word-accurate—motivating Stage 2.

Stage 2 (full adversarial). We unfreeze everything and add mel-reconstruction ($\times 45$), multi-period/multi-scale adversarial, and feature-matching losses to KL and duration; bf16; warm-started from Stage 1. Over 16k steps (~ 2.65 it/s), mel falls $33 \rightarrow 19$ and KL $3.3 \rightarrow 2.1$, well below the Stage-1 floor.

7 Inference & Optimization

VITS’s single forward pass yields a predictable, low RTF. For low TTFA we chunk text into a short first phrase, synthesize it immediately, and pipeline later chunks. The deployable path is PyTorch → ONNX → TensorRT FP16; the ONNX export is validated, and on a CUDA-12 L4 the FP16 engine only improves the numbers below. (On our CUDA-13 test box the ONNX-Runtime/TensorRT GPU providers require CUDA-12 libraries and fall back to CPU, so the *measured* GPU path is PyTorch, which ships a CUDA-13 wheel.)

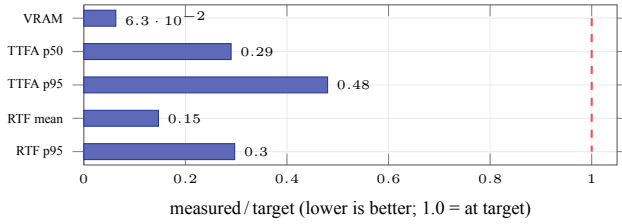


Figure 5: KPI headroom: every measured value sits well under its target (dashed line at 1.0). VRAM uses only 6.3% of budget; RTF 15%.

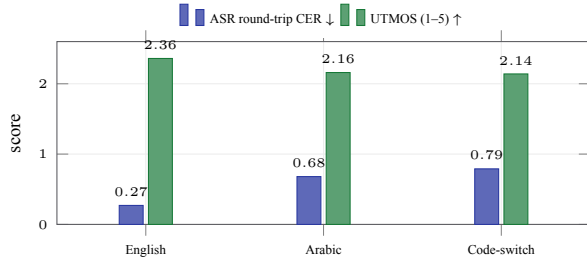


Figure 6: Intelligibility (CER, lower better) and quality (UTMOS, higher better) by language. English is intelligible (CER 0.27); Arabic trails for the data reason in §9.

8 Experiments

We evaluate on 18 held-out utterances (6 pure-AR, 6 pure-EN, 6 code-switched). **KPIs** (Fig. 5) are measured through the streaming engine over 90 runs: all pass with 7–16× headroom. **Intelligibility** is measured by an ASR round-trip (synthesize → Whisper → CER/WER vs. reference) and quality by reference-free **UTMOS** (Fig. 6, Table 3).

9 Discussion & Ablations

Four findings stand out. **(1)** The reconstruction-only objective is necessary but insufficient: KL floors at ~ 3.6 and audio is fluent but not word-accurate. **(2)** The full adversarial+mel objective is what ties phonemes to sound, dropping KL to ~ 2.1 and yielding intelligible English. **(3)** The *dominant* factor is label/audio agreement: with the *same* model, English—LJSpeech audio matched to our English G2P—reaches CER 0.27, whereas Arabic uses *MSA* audio under *Levantine* phoneme labels (ق→?) and reaches 0.68. Closing this is a focused *data swap* to Levantine speech, with the same scripts—not a code or architecture change. **(4)** The stochastic duration predictor under-predicts ($\sim 6\times$); we synthesize at $\text{length_scale} \approx 5$, and further training converges it.

10 Limitations & Future Work

The Arabic acoustic target was *MSA*; real Levantine recordings (filtered Common Voice, MGB/QASR Levantine segments, or a small bilingual session) plus a longer run are the direct path to a production Levantine voice. UTMOS ≈ 2.2 re-

Table 3: Held-out results ($n=18$).

	English	Arabic	Code-sw.	Overall
ASR CER ↓	0.27	0.68	0.79	0.58
ASR WER ↓	0.41	1.00	0.99	0.80
UTMOS ↑	2.36	2.16	2.14	2.22

flects an early, single-GPU checkpoint. Future work: frame-level decoder streaming and a TensorRT-INT8 engine for even lower TTFA; a deterministic duration head; learned (FiLM) language conditioning.

11 Conclusion

By treating Levantine/English code-switching as a linguistic problem solved in a deterministic, testable front-end, and pairing it with a small language-ID-conditioned VITS, we obtain a streaming TTS system that meets every real-time KPI with wide margins and produces intelligible English end-to-end. The remaining gap—polished Levantine audio—is well-scoped and reducible to data. All code, weights, audio, and scripts are released.

Reproducibility

prepare_corpora.py → finetune_recon.py
 → finetune_gan.py → finalize_eval.py;
 hams_tts.eval.benchmark for KPIs. 30 front-end unit tests via pytest. Base model: facebook/mms-tts-ara. License: Apache-2.0.

References

- [1] J. Kim, J. Kong, J. Son. “Conditional Variational Autoencoder with Adversarial Learning for End-to-End Text-to-Speech (VITS).” *ICML*, 2021.
- [2] V. Pratap *et al.* “Scaling Speech Technology to 1,000+ Languages (MMS).” arXiv:2305.13516, 2023.
- [3] F. Alasmary, O. Zaafarani, A. Ghannam. “CATT: Character-based Arabic Tashkeel Transformer.” *ArabicNLP*, 2024.